

Ingeniería de Software II

Evaluación de arquitecturas

Actividad 1

Cada grupo trabaja sobre su producto

Consigna:

- Definir un pipeline dataflow interno.
- Seleccionar un estilo distribuido (broker, pub-sub, etc.) para la comunicación.
- Seleccionar manejo de eventos

Dataflow

- ▶ Dominio que define como los datos fluyen por el sistema y son procesados / transformados.
- ▶ Estilos
 - Batch Secuencial
 - Pipes & Filters
 - Hierarchical Layers

Sistemas Distribuidos

- ▶ Dominio que define cómo se descubren miembros de la red y cómo se comunican.
- ▶ Estilos
 - Broker
 - Publish – Subscribe
 - Forwarder – Receiver
 - Client – Dispatcher – Server

Event Based Systems

Puede tomar múltiples flavors:

- Single Event Processing
- Event Stream Processing
- Complex Event Processing
- Online Event Processing

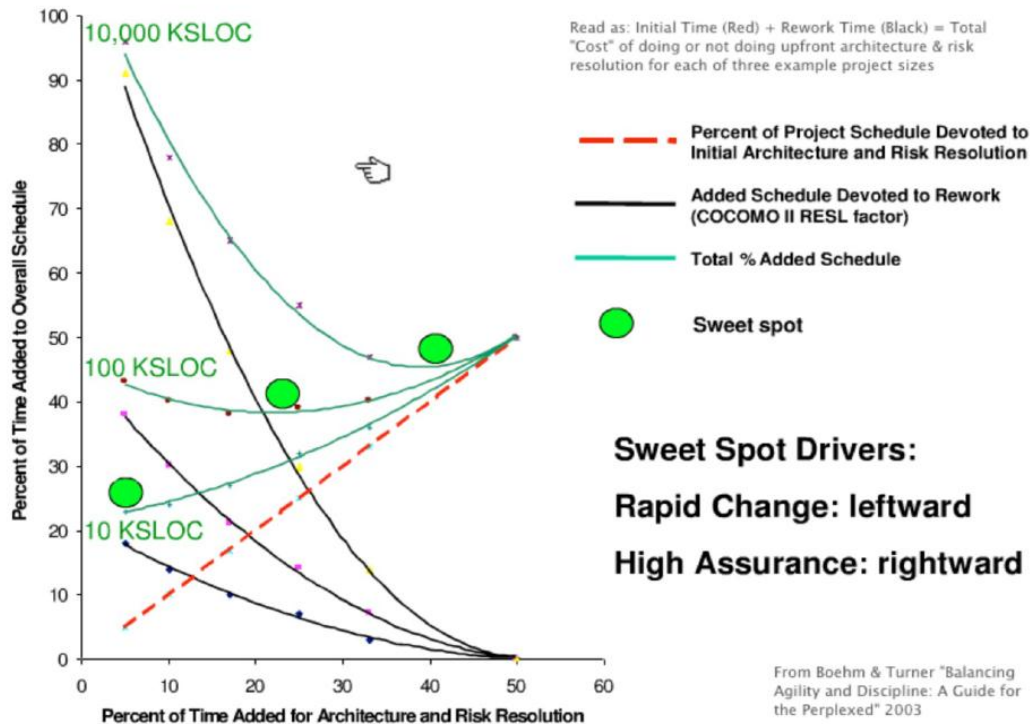
Arquitectura en waterfall y en agile

- Waterfall: BDUF ("big design up front"). Inversión significativa en etapas iniciales (blueprint, phase 0)
- Agile: YAGNI ("you are not gonna need it"). Refactoring para evolucionar los problemas y eliminar la deuda técnica que provocan las iteraciones
- Trade off entre *adaptación* y *anticipación*



Krutchen, 2010

Cuando definir la arquitectura?



Cuanto mas tardamos con respecto a lo que estimamos inicialmente

Cuanto tiempo agregamos al principio para arquitectura y contener riesgos

From Boehm & Turner "Balancing Agility and Discipline: A Guide for the Perplexed" 2003

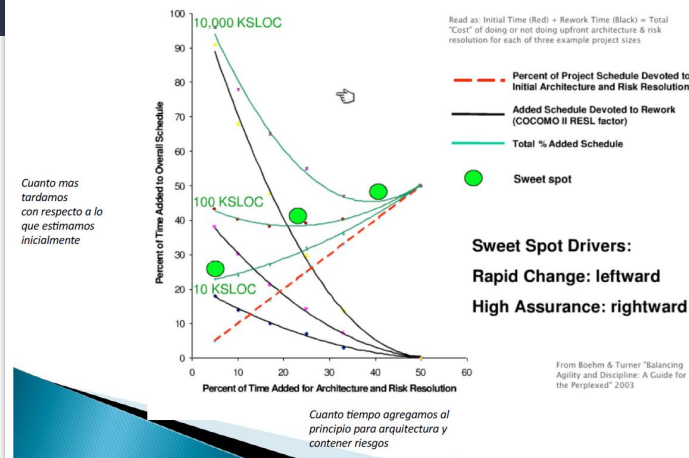
Diferencia según tamaño del proyecto

- 10.000 LOC: El punto óptimo está muy a la izquierda. Conviene **dedicar poco tiempo a arquitectura** (porque el proyecto es chico, los riesgos son manejables).
- 100.000 LOC : El sweet spot está en un punto intermedio: hay que balancear, **dedicar una proporción moderada de tiempo upfront**.
- 10 millones LOC: El óptimo se va mucho a la derecha: se necesita **invertir bastante tiempo inicial en arquitectura** y gestión de riesgos, porque el rework sería carísimo.

Conclusión

- El **momento y el esfuerzo** para definir la arquitectura dependen del **tamaño** del proyecto y del **nivel de certeza requerido**.
- En proyectos **chicos** → más **agilidad**, **menos** arquitectura **upfront**.
- En proyectos **grandes** o críticos → más **inversión inicial en arquitectura** y resolución de riesgos.

Cuando definir la arquitectura?



Scrum: cuando se define la arquitectura?

Cuatro posibilidades:

- Big up-front: antes de los sprints. En los sprints se hacen cambios menores. Tipicamente desarrollada por un arquitecto. Evaluación antes de los sprints
- Sprint-zero: durante el primer sprint (2 a 4 semanas). Tipicamente elaborada por el equipo de desarrollo. Evaluación al final del sprint.
- In-sprints: diseñada y refactorizada de acuerdo a la necesidad. Requiere un equipo de desarrollo maduro con mucha experiencia en el dominio. Evaluación luego de refactorizaciones significativas
- Separate architecture team: analiza ASRs y diseña para cada release. Evaluación luego de cada release significativo



Requisitos Arquitectónicamente Significativos (ASR):
RQFy RQNF que tienen un impacto directo y medible
en el diseño de la arquitectura

ASR

Un ASR es cualquier **requisito** (funcional o de calidad) cuya satisfacción obliga a **tomar decisiones arquitectónicas** clave.

Si **cambiaras ese requisito**, probablemente **cambiaría la arquitectura** del sistema.

Ejemplos de ASR

- Requisitos de calidad (no funcionales):
 - Escalabilidad: “El sistema debe soportar 1 millón de usuarios concurrentes”.
 - Seguridad: “Debe cumplir con GDPR y encriptar datos sensibles en tránsito y en reposo”.
 - Disponibilidad: “El sistema debe tener 99,999% de uptime”.
- Requisitos funcionales que afectan arquitectura:
 - “El sistema debe integrarse en tiempo real con sistemas externos de pago”.
 - “Debe permitir plug-ins de terceros sin redeployar el core”.

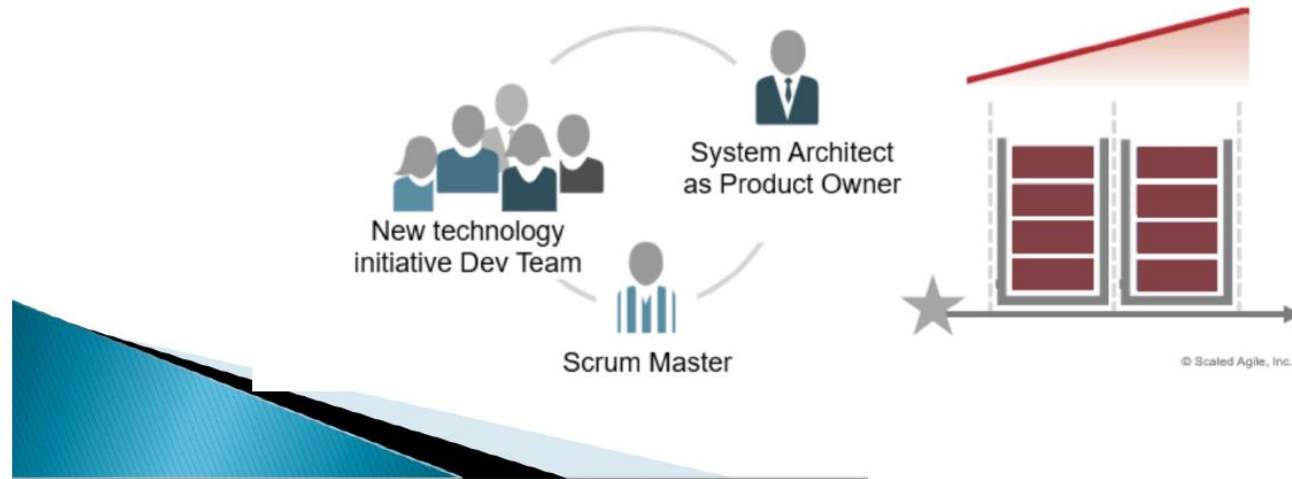
Ejemplo tradeoff: Agile Architecture en SAFe

- SAFe = Scaled Agile Framework (© Scaled Agile, Inc.)
- Diseñado para desarrollos de software grandes, típicamente corporativos
-> alta complejidad, muchos equipos
- Con su concepto de Agile Architecture, intenta balancear BDUF y YAGNI
 - Intentional architecture: estrategias planeadas de arquitectura, alineadas a los atributos escogidos, que proveen guías para sincronizar los diferentes equipos
 - Emergent design: evolución incremental de la implementación, que permite a desarrolladores responder inmediatamente a las necesidades de los usuarios



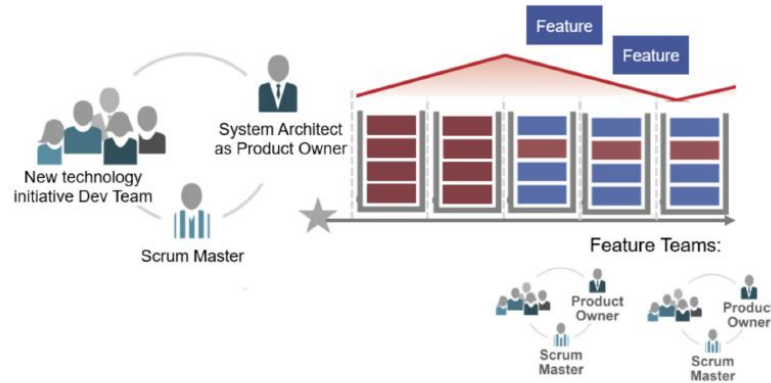
Agile Architecture en SAFe

- Architectural Runways: componentes, código e infraestructura necesaria para implementar los features en el corto plazo, sin necesidad de rediseño adicional
- Es diseñada en un conjunto reducido de iteraciones, con el arquitecto como product owner



Agile Architecture en SAFe

- Durante las iteraciones, los equipos usan el architectural runway...



- ... que con el tiempo se consume, por evolución de los requerimientos, por lo que debe ser mantenido / renovado periódicamente

Ejemplo: Si una empresa sabe que en el próximo trimestre deberá habilitar pagos internacionales, una parte del architectural runway puede ser diseñar e implementar servicios de seguridad, escalabilidad y compliance antes de que lleguen esas features. Así, cuando se prioriza la funcionalidad, ya hay una base que lo soporta sin frenar el delivery.

Architectural runway

- Incluye **arquitectura emergente** (lo que se va construyendo **sprint a sprint**) y **arquitectura intencional** (lo que **se planifica** a propósito para soportar el crecimiento).
- Architectural Runway != diseñar todo al inicio.
- Se trata de anticipar lo necesario a corto/mediano plazo para **no frenar el desarrollo, pero sin intentar adivinar todo** el futuro del sistema.
- Está compuesto por **componentes** reutilizables, **servicios** compartidos, **infraestructura** de integración y despliegue, **APIs** y tecnologías ya adoptadas.
- No es estático: **se amplía y ajusta continuamente** en sincronía con el avance de los Product Increments (PIs).

Evaluación de arquitecturas- Por que evaluar

- Para asegurar la robustez del diseño
- Con una evaluación podemos
 - Utilizar métodos repetibles, bien estructurados que pueden reducir y/o mitigar los riesgos técnicos del producto en forma temprana
 - Asegurar que una arquitectura es correcta (alineamiento a los requerimientos no funcionales)

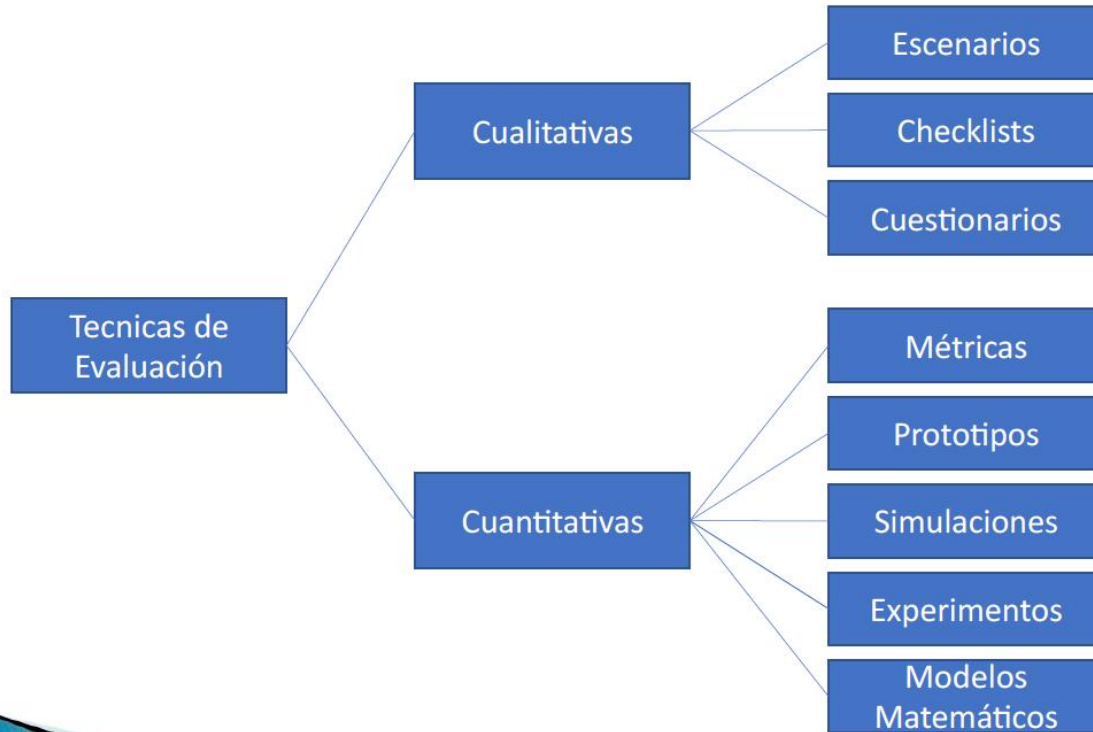


Técnicas de evaluación

- Se espera que el resultado de la evaluación responda
 - Si la arquitectura es apropiada para los requerimientos planteados
 - Si hay mas de una arquitectura candidata, cual es la adecuada
 - Indicar en que grados se cumplen los atributos de calidad
 - Detectar riesgos potenciales



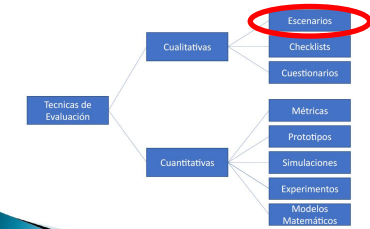
Técnicas de evaluación



Escenarios

- Breve **narrativa** de una situación potencial que debe enfrentar el sistema, desde el punto de vista de un stakeholder (usuario final, desarrollador, devops, etc.)
- **Que pasa si...**
 - Dos usuarios quieren comprar el mismo item?
 - 5000 usuarios se conectan a la vez al sistema?
 - Se corta internet?

Técnicas de evaluación



**MODELO DE
ESCENARIOS
“LIVIANO”**

Referencia: Scenario-Based Analysis of Software Architecture,
Kazman et al.

Checklists / Cuestionarios

- Lista de **criterios** que la arquitectura debe cumplir, que un evaluador debe calificar (checklists) o responder (cuestionarios)

Técnicas de evaluación



EA Design Review Checklist			Developed Solutions	Licensed Solution - Harvard's Cloud	Licensed Solution - Vendor Hosted	Subscription - SaaS
Version 2.2						
F3	Security	The solution MUST integrate with Harvard's single-sign-on authentication service HarvardKey, using either CAS, SAML2, or other supported protocols.	☐	☐	☐	☐
F4	Security	Mobile solutions, either stand-alone or adjuncts to larger solutions, MUST integrate with Harvard's single-sign-on authentication service HarvardKey, using either CAS, SAML2, or other supported protocols.	☐	☐	☐	☐
F5	Security	The solution MUST provide role-based authorization to users of the solution, either as a feature of the application or through an authorization service such as Group Services, in accordance to the principle of least privilege.	☐	☐	☐	☐
F6	Security	Solutions MUST conform to the appropriate HUIT security policies (https://policy.security.harvard.edu/security-requirements). Consult with your ISO for guidance.	☐	☐	☐	☐
F7	Security	The solution MUST constrain the permitted network, service, and package interactions that are allowed, in accordance to the principle of least privilege (for example DDNS protection, port visibility black/white listing).	☐	☐	☐	☐
F8	User Experience	Applications MUST meet the HUIT accessibility policy (http://accessibility.huit.harvard.edu/huit-policy).	☐	☐	☐	☐
F9	User Experience	The solution SHOULD work with both computers and mobile devices. Web-based solutions should work with a variety of industry-standard web browsers.	☐	☐	☐	☐
F10	User Experience	The solution SHOULD provide a modern user experience where the interface and workflow designs provide access to desired outcomes with the least friction. https://projects.iq.harvard.edu/harvarduxgroup/home	☐	☐	☐	☐

Fuente: <https://enterprisearchitecture.harvard.edu/application-architecture-checklist>

Métricas

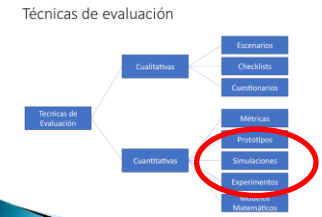
- Availability: SLA, SLO, SLI
- Reliability, Fault Tolerance: # of failures, MTBF, MTTR
- Performance: response time
- Scalability: # of concurrent users
- Maintainability: lead time for changes, MTTR
- Security: # of security incidents

Técnicas de evaluación



SLI – Service Level Indicator: métrica concreta y cuantificable
SLO – Service Level Objective: objetivo o meta que fijás para un SLI
SLA – Service Level Agreement: acuerdo formal/contractual entre proveedor y cliente (SLO + penalidades)

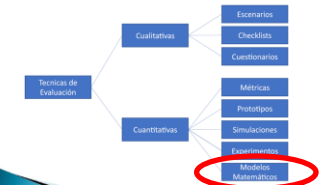
Prototipos / Simulaciones / Experimentos



- Simulaciones: software que permite diseñar (típicamente en un DSL) una arquitectura y evaluar su comportamiento frente a diferentes escenarios
Ejemplo: Palladio (<https://www.palladio-simulator.com/>)
- Experimento: software diseñado a probar una hipótesis específica sobre el comportamiento de la arquitectura
- Prototipo: software que constituye una versión preliminar de la arquitectura, que puede evolucionar a versiones posteriores

Modelos matemáticos

Técnicas de evaluación



- Software reliability growth models (SRGM): modelos de fallas y recuperación basados en análisis probabilístico de fallas y distintos esquemas de interacción entre los componentes del software
- Software Architecture-based Performance Analysis: análisis de la respuesta en throughput, utilización de recursos o tiempo de respuesta
- La complejidad y esfuerzo requeridos justifica su empleo en casos de alta criticidad: sistemas médicos, aviones, misiones espaciales

CARO ++++

Metodologías basadas en escenarios

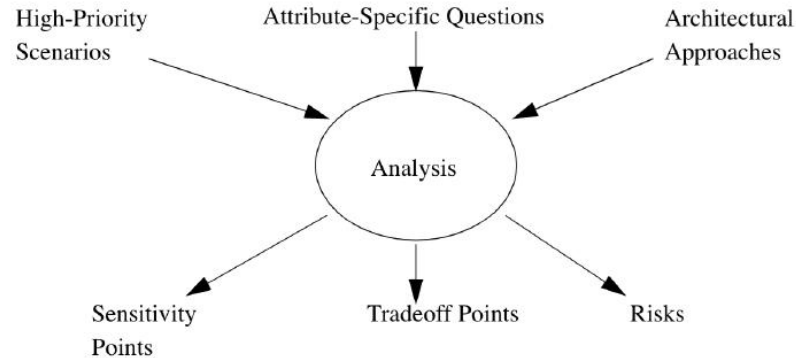
- SAAM (Scenario-based Software Architecture Analysis Method) (1993): verificar supuestos básicos de arquitectura contra los documentos principales que describen las propiedades deseadas del software.
- ATAM (Architecture Trade Off Analysis Method) (2000): foco en como compiten distintos atributos para lograr la arquitectura deseada.
- Es el “gold standard” de las metodologías de evaluación. Requiere esfuerzo significativo (20 a 30 días/hombre)
- Lightweight ATAM: versión recortada, puede realizarse en un día o medio día. El foco es similar a ATAM
- No son las únicas; existen numerosas instancias de revisión de acuerdo a la metodología usada



**MODELO DE
ESCENARIOS
“PESADO”**

ATAM

- Architecture Tradeoff Analysis Method
- Diseñado por el SEI (Software Engineering Institute – Carnegie Mellon University)
- Utiliza cuestionarios y escenarios
- Se enfoca en los atributos de calidad
- Esencialmente cualitativo, genera tendencias



Atributos de calidad – Repaso

FUNCIONAMIENTO

- Rendimiento (Performance): tiempo de respuesta, throughput, latencia.
- Disponibilidad (Availability): tiempo en línea, tolerancia a fallos, redundancia.
- Seguridad (Security): confidencialidad, integridad, autenticación, autorización.
- Escalabilidad (Scalability): capacidad de crecer en volumen de usuarios o datos.
- Interoperabilidad (Interoperability): integración con otros sistemas.
- Administrabilidad (Manageability): facilidad para configurar, monitorear y operar el sistema en producción.
- Confiabilidad (Reliability): capacidad de funcionar de manera consistente y recuperarse de fallos sin pérdida de datos ni interrupciones críticas.

USUARIO

- Usabilidad (Usability): facilidad de aprendizaje, eficiencia de uso.

DISEÑO

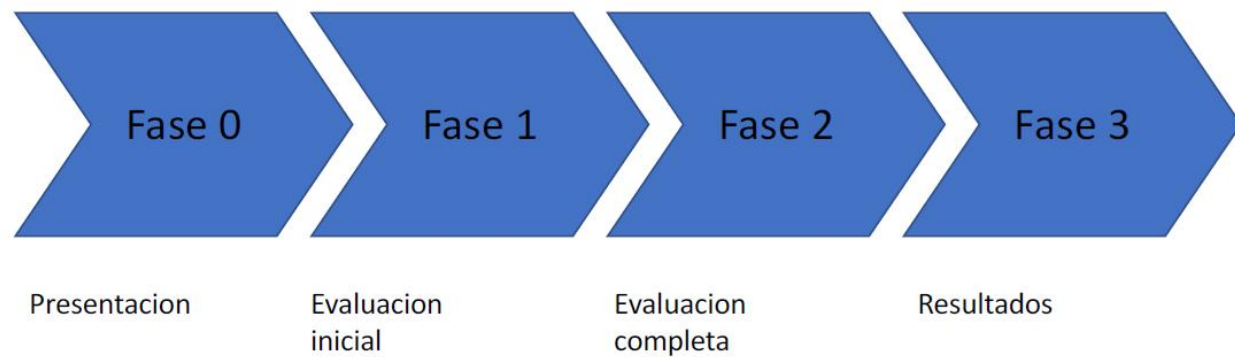
- Modificabilidad / Mantenibilidad (Modifiability): facilidad para cambiar, reparar o evolucionar el sistema.
- Portabilidad (Portability): facilidad para mover a otro entorno tecnológico.
- Reusabilidad (Reusability): posibilidad de aprovechar componentes en otros contextos.

SISTEMA

- Testabilidad (Testability): facilidad para validar requisitos.

*Ojo: Esto implementa
Parcializable*

ATAM-Fases



Fase 0

Kickoff

◦ Se acuerda la evaluación.

◦ Se presenta al equipo.

- Evaluadores → **equipo de expertos en arquitectura que conducen la evaluación**
- Redactores (notas, actas, escenarios, resultados) → **documentar todo lo que surge en la sesión.**
- Inquisidor → **hacer preguntas difíciles y profundas**
- Moderador (Process enforcer) → **Facilita la sesión para que sea productiva y ordenada**
- Decision makers (PM y Arquitecto) → **priorizar atributos de calidad y tomar decisiones sobre el alcance**
- Stakeholders → **aporta perspectivas diversas sobre el sistema**

Fases 1, 2 y 3

Ejecución

Fase 1

- Presentación
 - ATAM
 - Drivers del negocio
 - Arquitectura
- Investigación y análisis
 - Identificación de los enfoques arquitecturales
 - Generación del árbol de utilidad
 - Análisis de los enfoques arquitecturales

Fase 2

- Testing
 - Brainstorming y priorización de escenarios
 - Análisis de las enfoques arquitecturales

Fase 3

- Reporting
 - Presentación de resultados

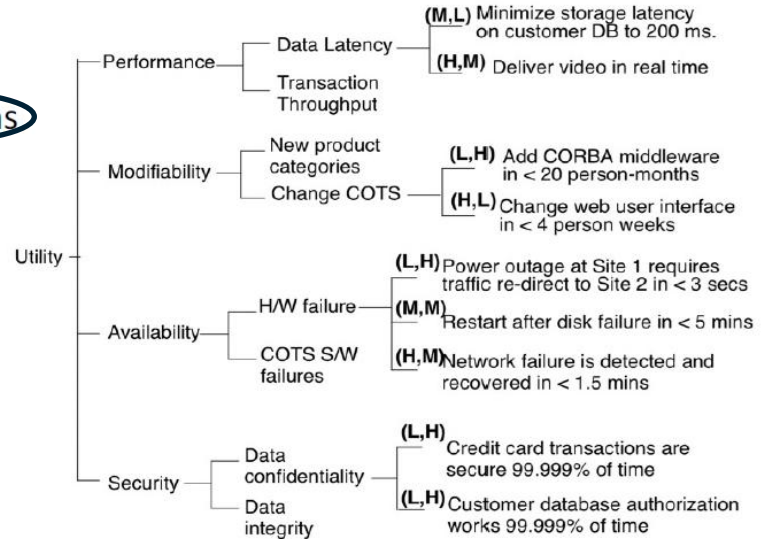
Fase 1- Arbol de Utilidad

- Sirve para enfocar el trabajo a realizar

- Atributos de calidad como nodos de alto nivel
- Nodos intermedios como características (features) que contribuyen al atributo
- Hojas como escenarios

- Anotar las hojas [importancia , dificultad]

- Importancia (alta, media o baja) para los stakeholders
- Dificultad (alta, media o baja) para lograr el objetivo



Fase 2- Brainstorming y priorización de escenarios

- Participan los stakeholders
- Se generan nuevos escenarios
 - Brainstorming
- Se priorizan los escenarios
 - Por votación
 - Votan los stakeholders con un número fijo de votos



**MODELO DE
ESCENARIOS
“LIVIANO”**



Fase 2- Análisis de los enfoques arquitecturales

- Identificación del enfoque
- Analizar los escenarios del árbol de utilidad
- Reportar y notificar

- Riesgos
- No riesgos
- Puntos sensibles
- Trade offs

Hint: “no requiere...”, “alcanza con...”, “se podrán cambiar...”

De los atributos de calidad



Fase 3 – Resultados

- Se genera el reporte final
 - Resumen
 - Descripción de ATAM
 - Descripción de los business drivers y arquitectura
 - Lista de los escenarios de la fase 1 y 2
 - El árbol de utilidad
 - Análisis de la fase 1 y 2
 - Enfoques arquitectónicos
 - Decisiones
 - Riesgos y no riesgos, puntos sensibles y tradeoff



Lightweight ATAM

- Desarrollado por el mismos autores que ATAM, para tener una alternativa de costo/beneficio razonable
- Util para revisiones de QA intermedias
- Puede ser realizada integramente con el equipo del proyecto. Esto acelera el tiempo de startup
- No hay reporte final, solo una minuta
- Contras:
 - Análisis de menor profundidad
 - Menor cantidad de ideas innovativas
 - Menor objetividad



Lightweight ATAM

Paso	Tiempo	Descripción
Presentar ATAM	0 horas	Omitido
Presentar Business Drivers	0,25 horas	Los participantes conocen el contexto
Presentar Arquitectura	0,5 horas	Vistas básicas de la arquitectura, con uno o dos escenarios ejecutados sobre las mismas
Identificar Enfoques de Arquitectura	0,25 horas	Solo se analizan enfoques relacionados con atributos de calidad específicos
Generar Arbol de Utilidad	0,5 a 1,5 horas	Debería existir un árbol de utilidad previo, y escenarios previos, en caso contrario puede tomar mas tiempo
Analizar Enfoques de Arquitectura	2 a 3 horas	Mapear los escenarios de mas alta prioridad en la arquitectura. Es la tarea mas importante
Brainstorming y priorización de escenarios	0 horas	Omitido, los escenarios se analizan al generar el arbol de utilidad
Analizar Enfoques de Arquitectura	0 horas	Omitido
Presentar Resultados	0,5 horas	Enfoque resumido, sin un reporte formal
Total	4 a 6 horas	

Reflexión final

- YAGNI vs BDUF: tradeoff
- La arquitectura debe evaluarse
- De acuerdo al proyecto y a la metodología usada, se debe escoger el momento y la técnica a aplicar
- ATAM sigue siendo la técnica de referencia para evaluar arquitecturas...
- ... pero las metodologías ágiles utilizan ceremonias y técnicas mas ligeras para evaluar las arquitecturas emergentes



Actividad 2

Definiciones de Arquitectura

- Documentar atributos de calidad prioritarios en un árbol de utilidad (atributos y escenarios).
- Documentar al menos 2 vistas del Modelo 4+1 (1 o 2 diagramas por cada una)
- Incorporar los estilos de arquitectura definidos en la clase pasada (Actividad 1)

